Controlling for Network Biases

Data collection biases are a persistent issue in studies of social networks. Two main types of biases can be considered: exposure biases and censoring biases .

To account for exposure biases, we can switch the network link probability model from a *Poisson* distribution to a *Binomial* distribution, as the binomial distribution allows us to account for the number of trials for each data estimation.

To address censoring biases, we need to add an additional equation to account for the probability of missing an interaction during observation when modeling the interaction between individuals i and j.

Considerations

> Caution

Example 1

Below is an example code snippet demonstrating a Bayesian network model with a sender-receiver effect, a dyadic effect, and a block model effect while accounting for exposure biases. This example is based on Sosa et al. (n.d.).

Python

```
from BayesForge import bf
import jax.numpy as jnp

# Setup device-------------------------------------------------
m = bf(platform='cpu', rand_seed=False)
```

```
# Simulate data ------------------------------------------------
N = 50
individual_predictor = m.dist.normal(0, 1, shape=(N, 1), sample=True)

kinship = m.dist.bernoulli(0.3, shape=(N, N), sample=True)
kinship = kinship.at[jnp.diag_indices(N)].set(0)

category = m.dist.categorical(jnp.array([.25, .25, .25, .25]), sample=True, shape=(N,))
N_grp, N_by_grp = jnp.unique(category, return_counts=True)
N_grp = N_grp.shape[0]

# Exposure: number of observation sessions of each dyad (unequal sampling effort)
exposure = m.dist.poisson(5, shape=m.net.mat_to_edgl(kinship).shape, sample=True) + 1

def sim_network(kinship, individual_predictor, category, exposure):
    B_intercept = m.net.block_model(jnp.full((N,), 0), 1, jnp.array([N]), sample=True)
    B_category = m.net.block_model(category, N_grp, N_by_grp, sample=True)
    sr = m.net.sender_receiver(individual_predictor, individual_predictor,
                               s_mu=0.4, r_mu=-0.4, sample=True)
    DR = m.net.dyadic_effect(kinship, d_sd=2.5, sample=True)
    # Number of sessions (out of `exposure`) in which the interaction was seen
    return m.dist.binomial(total_count=exposure, logits=B_intercept + B_category + sr + DR,
                           sample=True)

network = sim_network(m.net.mat_to_edgl(kinship), individual_predictor, category, exposure)

# Predictive model ---------------------------------------------
m.data_on_model = dict(
    network=network,
    exposure=exposure,
    dyadic_predictors=m.net.mat_to_edgl(kinship),
    focal_individual_predictors=individual_predictor,
    target_individual_predictors=individual_predictor,
    category=category
)

def model(network, exposure, dyadic_predictors, focal_individual_predictors,
          target_individual_predictors, category):
    N_id = focal_individual_predictors.shape[0]

    # Block ----------------------------------------
    B_intercept = m.net.block_model(jnp.full((N_id,), 0), 1, jnp.array([N_id]), name="B_inte
```

```
    B_category = m.net.block_model(category, N_grp, N_by_grp, name="B_category")

    ## SR shape = N individuals---------------------------------------
    sr = m.net.sender_receiver(focal_individual_predictors, target_individual_predictors,
                               s_mu=0.4, r_mu=-0.4)

    # Dyadic shape = N dyads-----------------------------------------
    dr = m.net.dyadic_effect(dyadic_predictors, d_sd=2.5)

    # Exposure: each dyad was observed `exposure` times (binomial instead of Bernoulli)
    m.dist.binomial(total_count=exposure, logits=B_intercept + B_category + sr + dr, obs=netw

m.fit(model, progress_bar=False)
m.summary()
```

R

```
library(BayesForge)

# Setup device--------------------------------------------------
m <- importBF(platform = "cpu", rand_seed = FALSE)
jnp <- reticulate::import("jax.numpy")

# Simulate data ------------------------------------------------
N <- 50L
individual_predictor <- m$dist$normal(0, 1, shape = reticulate::tuple(N, 1L), sample = TRUE)

kinship <- m$dist$bernoulli(0.3, shape = reticulate::tuple(N, N), sample = TRUE)
diag_idx <- jnp$diag_indices(N)
kinship <- kinship$at[reticulate::tuple(diag_idx[[1]], diag_idx[[2]])]$set(0L)

category <- m$dist$categorical(jnp$array(c(0.25, 0.25, 0.25, 0.25)), sample = TRUE, shape = 
u <- jnp$unique(category, return_counts = TRUE)
N_grp <- u[[1]]$shape[[1]]
N_by_grp <- u[[2]]

# Exposure: number of observation sessions of each dyad (unequal sampling effort)
edges <- m$net$mat_to_edgl(kinship)
exposure <- m$dist$poisson(5, shape = reticulate::tuple(edges$shape[[1]], edges$shape[[2]]),

sim_network <- function(kinship, individual_predictor, category, exposure) {
```

```r
  B_intercept <- m$net$block_model(jnp$full(reticulate::tuple(N), 0L), 1L, jnp$array(list(N)
  B_category <- m$net$block_model(category, N_grp, N_by_grp, sample = TRUE)
  sr <- m$net$sender_receiver(individual_predictor, individual_predictor,
                              s_mu = 0.4, r_mu = -0.4, sample = TRUE)
  DR <- m$net$dyadic_effect(kinship, d_sd = 2.5, sample = TRUE)
  # Number of sessions (out of `exposure`) in which the interaction was seen
  m$dist$binomial(total_count = exposure, logits = B_intercept + B_category + sr + DR, sample
}

network <- sim_network(m$net$mat_to_edgl(kinship), individual_predictor, category, exposure)

# Predictive model ------------------------------------------------
m$data_on_model <- list(
  network = network,
  exposure = exposure,
  dyadic_predictors = m$net$mat_to_edgl(kinship),
  focal_individual_predictors = individual_predictor,
  target_individual_predictors = individual_predictor,
  category = category
)

model <- function(network, exposure, dyadic_predictors, focal_individual_predictors,
                  target_individual_predictors, category) {
  N_id <- focal_individual_predictors$shape[[1]]

  # Block ----------------------------------------
  B_intercept <- m$net$block_model(jnp$full(reticulate::tuple(N_id), 0L), 1L, jnp$array(list
  B_category <- m$net$block_model(category, N_grp, N_by_grp, name = "B_category")

  ## SR shape = N individuals----------------------------------------
  sr <- m$net$sender_receiver(focal_individual_predictors, target_individual_predictors,
                              s_mu = 0.4, r_mu = -0.4)

  # Dyadic shape = N dyads-----------------------------------
  dr <- m$net$dyadic_effect(dyadic_predictors, d_sd = 2.5)

  # Exposure: each dyad was observed `exposure` times (binomial instead of Bernoulli)
  m$dist$binomial(total_count = exposure, logits = B_intercept + B_category + sr + dr, obs =
}

m$fit(model, progress_bar = FALSE)
m$summary()
```

Julia

```
using PyBayesForge
using PythonCall

# Setup device--------------------------------------------------
m = importBF(platform="cpu", rand_seed=false)

# Simulate data ------------------------------------------------
N = 50
individual_predictor = m.dist.normal(0, 1, shape=(N, 1), sample=true)

kinship = m.dist.bernoulli(0.3, shape=(N, N), sample=true)
kinship = kinship.at[jnp.diag_indices(N)].set(0)

category = m.dist.categorical(jnp.array([0.25, 0.25, 0.25, 0.25]), sample=true, shape=(N,))
u = jnp.unique(category, return_counts=true)
N_grp = u[0].shape[0]
N_by_grp = u[1]

# Exposure: number of observation sessions of each dyad (unequal sampling effort)
exposure = m.dist.poisson(5, shape=m.net.mat_to_edgl(kinship).shape, sample=true) + 1

function sim_network(kinship, individual_predictor, category, exposure)
    B_intercept = m.net.block_model(jnp.full((N,), 0), 1, jnp.array((N,)), sample=true)
    B_category = m.net.block_model(category, N_grp, N_by_grp, sample=true)
    sr = m.net.sender_receiver(individual_predictor, individual_predictor,
                               s_mu=0.4, r_mu=-0.4, sample=true)
    DR = m.net.dyadic_effect(kinship, d_sd=2.5, sample=true)
    # Number of sessions (out of `exposure`) in which the interaction was seen
    return m.dist.binomial(total_count=exposure, logits=B_intercept + B_category + sr + DR, s
end

network = sim_network(m.net.mat_to_edgl(kinship), individual_predictor, category, exposure)

# Predictive model ---------------------------------------------
m.data_on_model = pydict(
    network=network,
    exposure=exposure,
    dyadic_predictors=m.net.mat_to_edgl(kinship),
    focal_individual_predictors=individual_predictor,
    target_individual_predictors=individual_predictor,
```

```
    category=category
)

@BF function model(network, exposure, dyadic_predictors, focal_individual_predictors,
                   target_individual_predictors, category)
    N_id = focal_individual_predictors.shape[0]

    # Block ----------------------------------------
    B_intercept = m.net.block_model(jnp.full((N_id,), 0), 1, jnp.array((N_id,)), name="B_inte
    B_category = m.net.block_model(category, N_grp, N_by_grp, name="B_category")

    ## SR shape = N individuals----------------------------------------
    sr = m.net.sender_receiver(focal_individual_predictors, target_individual_predictors,
                               s_mu=0.4, r_mu=-0.4)

    # Dyadic shape = N dyads-----------------------------------------
    dr = m.net.dyadic_effect(dyadic_predictors, d_sd=2.5)

    # Exposure: each dyad was observed `exposure` times (binomial instead of Bernoulli)
    m.dist.binomial(total_count=exposure, logits=B_intercept + B_category + sr + dr, obs=netw
end

m.fit(model, progress_bar=false)
m.summary()
```

Example 2

Below is an example code snippet demonstrating a Bayesian network model with a sender-receiver effect, a dyadic effect, and a block model effect while accounting for exposure biases and censoring biases:

Mathematical Details

Main Formula

$$Y_{[i,j]} \sim \text{Binomial}\Big(E_{[i,j]}, Q_{[i,j]}\Big)$$

$$Q_{[i,j]} = \phi_{[i,j]}\eta_{[i]}\eta_{[j]}$$

Where:

- $E_{[i,j]}$ is the number of trials for each observation (i.e., the sampling effort).
- $Q_{[i,j]}$ is the indicator of a true tie between i and j, defined as:

$$Q_{[i,j]} \sim \begin{cases} 0 & \text{if no interaction occurs or if } i \text{ or } j \text{ is not detectable} \\ 1 & \text{if } i \text{ and } j \text{ are both detectable} \end{cases}$$

- $\phi_{[i,j]}$ is the probability of a true tie between i and j.
- $\eta_{[i]}$ is the probability of individual i being detectable.
- $\eta_{[j]}$ is the probability of individual j being detectable.

Defining formula sub-equations and prior distributions

We can let $\eta_{[i]}$ depend on individual-specific covariates. To model the probability of censoring, we can model $1 - \eta_{[i]}$:

$$\text{logit}(1 - \eta_{[i]}) = \mu_\psi + \hat{\psi}_{[i]}\sigma_\psi + ...$$

Where:

- μ_ψ is the intercept term.
- σ_ψ is a scalar for the variance of random effects.
- $\hat{\psi}_{[i]} \sim \text{Normal}(0, 1)$, and the ellipsis signifies any linear model of coefficients and individual-level covariates. For example, if C is an animal-specific measure, like a binary variable for cryptic coloration, then the ellipsis may be replaced with $\kappa_{[5]}C_{[i]}$ to give the effects of coloration on censoring probability.

Model validation tests

- Documentation examples: the Python, R and Julia examples of this page are run by the documentation tests (`Python/test_model_pages.py`, `R/test_r_model_pages.py`, `Julia/test_julia_model_pages.py`).

Note(s)

> **Note**
>
> - One major limitation of this model is the necessity of having an estimation of the censoring bias for each individual.

Sosa, Sebastian, Mary B. McElreath, Daniel Redhead, and Cody T. Ross. n.d. "Robust Bayesian Analysis of Animal Networks Subject to Biases in Sampling Intensity and Censoring." *Methods in Ecology and Evolution*, 1–22. https://doi.org/https://doi.org/10.1111/2041-210X.70017.